# Whole-Route Planning of a Memory Bus Between Two BGAs: An Exact Model of Crossings and Layer Changes, Negotiated to DRC-Clean Copper

Andrew Haas

Abstract—Routing the bus between a processor and its DRAM remains largely manual in printed circuit board (PCB) design, and published algorithms optimize proxies such as crossing or layer counts at the planning stage. We present a router that plans every lane of a two-layer BGA-to-BGA bus completely before any copper is laid, and carries the plan to finished, design-rule-clean copper. Its core is an exact constraint-programming model that chooses, for all lanes at once, where each necessary crossing occurs, the order of the crossings (consistent with a wiring diagram of the lanes' permutation by a rule over every triple of lanes), and where each lane changes layer, with via room and spacing; for the ends it is given, it proves the fewest layer changes within the model. The model reads surprisingly naturally as a problem in physics—worldlines, frustrated Ising spins and contact forces—and the connections proved of practical use. A linear program then places the lanes in millimeters; the constraints it must violate return to the model as cuts, and audit findings as negotiated congestion prices, until the plan passes every rule of the production router. The escapes at both arrays are chosen by an estimate of the same objective, and differential pairs are planned with their polarity at each end, with an automatic via crossover where the ends disagree. On two public boards, an Allwinner H3 and a Zynq-7020 each to a DDR3 device, the router completes every rung of a ladder of up to 51 nets connected and DRC-clean, never with more vias than the human designer (80 against 88 on the H3, 58 against 103 on the Zynq) and always with less copper.

Index Terms—Printed circuit board, bus routing, escape routing, topological via minimization, layer assignment, constraint programming, linear programming, differential pairs, DDR.

## I. Introduction

A memory bus is the densest wiring on most processor boards: tens of nets leave a ball grid array (BGA) in a fixed order, arrive at another in a different order, and must cross one another on the way. On a two-layer region every crossing forces one of the two nets onto the other layer there, and every layer change is a via—a drill, an annular ring that blocks both layers, and a discontinuity in the signal path. Commercial tools approach it interactively: the designer sketches the flow, and the tool fills it in.

The literature treats the problem in stages—escape routing, simultaneous escape routing, bus planning and layer assignment (Section II)—each optimizing a proxy such as crossings, layers or a planned via count, evaluated against other planners on industrial cases that cannot be reproduced. Whether a plan can be laid in copper at the board's clearance, and how its via count compares with a designer's, are left open.

This paper starts from one observation: the ends decide the braid. Once each net's escape at the source and destination is fixed, the orders of those escapes around the two arrays fix which pairs of nets must cross. What remains free is where each crossing happens, in what order the crossings occur, and where each lane changes layer. These choices interact—a via placed to serve one crossing serves or spoils the next, and two vias need room from each other and from every lane passing by—so we decide all of them at once, for every lane, before anything is routed. The contributions are:

1) An exact constraint-programming (CP) model of the whole route (Section IV) that chooses crossing positions, their order, and every lane's layer changes jointly. Its consistency rule over triples of lanes admits exactly the orders that a wiring diagram of the lanes' permutation can realize. For the ends it is given, the model proves the fewest layer changes within its scope.
2) A realization of the plan in board geometry (Section VI): an elastic linear program whose violated constraints return to the CP model as cuts, a snap onto the production router's grid, and an audit against the router's own rules, closed in a negotiation loop that also feeds findings at the ends back to the choice of escapes (Section V).
3) Differential pairs planned as single lanes with a polarity (handedness) at each end, and an automatic via crossover that swaps P and N where the two ends disagree.
4) An evaluation on two public boards against the human layouts, graded on finished copper by independent design-rule and connectivity checks (Section VII).

The models have a surprisingly natural reading in physics. Read along the route, each lane is the worldline of a particle in one space and one time dimension, and each crossing a scattering event; the triple rule that keeps the crossings consistent is the Yang–Baxter relation of three strands. Each worldline carries an Ising spin, its layer,

A. Haas is a Professor in the Department of Physics, New York University, New York, NY 10003 USA (e-mail: andy.haas@nyu.edu).

The software, the benches and the recipes that reproduce every number in this paper are open source [1].

which must disagree with its partner's at every crossing: a via is a domain wall, and frustration is what forces vias. Once the discrete choices are made, placing the lanes is a problem in statics—strings under tension, held apart by struts and walls—whose contact forces are the linear program's dual variables.

## II. Related Work

Topological via minimization. Given the nets' topology, choosing layers to minimize vias is classical. Hsu [2] decoupled topology from geometry to minimize vias; Marek-Sadowska [3] showed the unconstrained problem NP-complete, and Sarrafzadeh and Lee [4] gave a polynomial algorithm for nets between two shores, where the problem reduces to covering a permutation graph. Constrained via minimization assigns layers to an already routed layout [5]: it is NP-complete in general [6], and where the layout's junctions have degree at most three it is a maximum cut on a planar graph, solvable in polynomial time [5], [7], which Barahona et al. solved with the same code as Ising spin-glass ground states [8]. Neither form chooses where the crossings occur, or gives vias their room, jointly with the layers.

PCB escape and bus routing. Ozdal and Wong chose escape patterns at both ends of a channel jointly, with a longest-path search for each layer's planar subset and negotiation for the rest [9], and Ozdal, Wong and Honsinger minimized the vias the choice causes directly [10]. Bus planning and layer assignment for dense boards followed [11], [12], as did exact satisfiability models for escape routing at one array [13], [14] and escape of differential pairs [15]. Most recently, Wu et al. [16] unified the escape and area regions in one circle model, searched escape orders by simulated annealing, and split unassigned nets with vias, reporting 40.9% fewer planned vias than a two-phase baseline. Apart from the escape routers, which lay detailed escapes at one array [13], [14], this work stops at a plan and is evaluated against other planners on private cases.

Topology, then geometry. Rubber-band routers [17] and the theory of homotopic routing [18] separate a topological route from its embedding. Negotiated congestion [19] prices places that were overused before. Our loop resembles logic-based and combinatorial Benders decomposition [20], [21], but its cuts are heuristic generalizations of a geometric failure, not proofs (Section VI-E).

Wiring diagrams. The orders in which the crossings of a permutation's reduced words can occur are the convex orders of its inversion set [22]; for the longest permutation their commutation classes form the higher Bruhat order $B(n, 2)$ [23]. We are not aware of a model that uses this structure to minimize vias jointly with crossing positions and via room.

## III. The Problem and the Frame

### A. Setting

A bench is a board region holding a source BGA $S$ (the processor) and a destination BGA $D$ (the DRAM), a set of bus nets between them, and everything else—passives, other nets' stubs, the board edge—as fixed obstacles. The region has two signal layers, F and B. Each net leaves $S$ at a tooth, the end of a fanout stub from its ball, and arrives at $D$ at a berth; each end has a layer. The fanout engine offers every ball a menu of escape moves (a stub on one layer, or a dog-bone via to the other), from which the ends are chosen (Section V). A differential pair is one lane with two legs. A lane's layer change is a dive: one via for a single lane, two barrels side by side for a pair. We route at 0.1 mm clearance with 0.127 mm lanes, 0.1 mm fanout stubs and 0.25 mm vias (0.15 mm drill) on a 0.025 mm grid. Two neighboring lanes on one layer must lie at least a bar of 0.232 mm apart, center to center.

The objective is lexicographic: first, the vias beyond two on each net, summed over the nets (its stubs' vias included, a pair counted by its worse leg); second, the lanes' layer changes; third, a congestion tie-break. The first two terms count planned vias, not the board's: the stub vias come from the choice of ends (Section V), and a pair's layer change counts once although it lays two barrels. The result must be routed by the production router, connected, and clean under a design-rule check (DRC) at the board's clearance.

### B. The frame and the route coordinate

The frame is read off the board alone. A straight trunk runs from the pad box of $S$ through that of $D$; a ring follows the octilinear hull of $D$'s pads and stubs around each of its two long faces. Unrolling the perimeter of $S$ from a cut on its far face gives the launch order $\lambda$ of the teeth; unrolling that of $D$ from a cut at the widest gap on its far face gives the berth order $\varphi$. Every lane's route is then one scalar coordinate $u$, arclength along a spine shared by all lanes: the trunk's straight spine, continued past each ring's handoff by arclength along that ring's own spine. Lanes at the same $u$ stand across the bus from one another, so a crossing's position is one number for both lanes. We discretize $u$ on a grid $G$ of a fifth of the least planned lane pitch $p = 0.252$ mm, about 0.05 mm.

A pair of lanes $(a, b)$ with $\lambda(a) < \lambda(b)$ is inverted if $\varphi(a) > \varphi(b)$. Every inverted pair must cross; in our model it crosses exactly once, and no other pair crosses. The set $I$ of inverted pairs is the inversion set of the permutation $\pi$ that takes launch order to berth order (Fig. 1). We call the crossings, with the order in which they occur, the braid; unlike a braid in topology it records no over and under, which is what the layers decide.

### C. Two bounds

Lemma 1 (Floor): If both ends of every lane are on F, any routing within the frame—lanes that pass round neither array, so that they leave the source in the order $\lambda$ and reach the destination in the order $\varphi$—needs at least $2(K - \mathrm{LIS}(\pi))$ vias, where LIS is the length of the longest increasing subsequence.

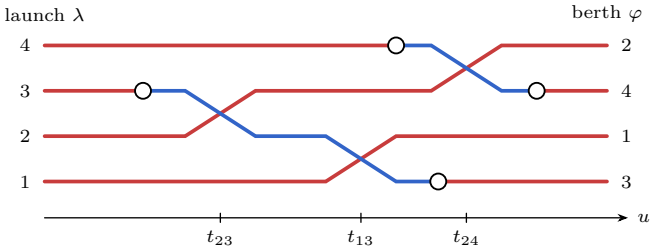



Fig. 1. Four lanes as a wiring diagram along the route coordinate $u$. The berth order is $\pi = (3, 1, 4, 2)$, so the inverted pairs $\{2,3\}, \{1,3\}, \{2,4\}$ must each cross once. On the triple $(1, 2, 3)$ only $\{1,3\}$ and $\{2,3\}$ are inverted, so the triple rule forces $t_{23} < t_{13}$. Red is F, blue is B; circles are vias. Crossing lanes are on different layers. Lane 3 serves its two F partners in one excursion to B, and lane 4 serves one: four vias, equal to the floor $2(K - \mathrm{LIS}(\pi)) = 2(4 - 2)$ of Lemma 1.

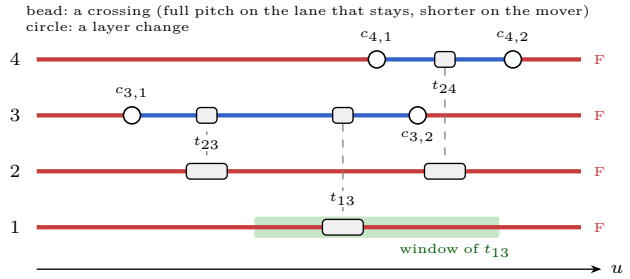



Fig. 2. The whole-route model as beads on tracks, for the four lanes of Fig. 1 (all ends on F). Each lane's route coordinate $u$ is a track, drawn in launch order. A crossing variable $t_{ab}$ is one bead that sits on both lanes' tracks at once (dashed tie), and intervals on one track may not overlap; each crossing slides only within its window (green, shown for $t_{13}$). Layer changes $c_{n,k}$ are beads too (circles), and between them a track's color is its layer. The model requires tied tracks to differ in color at their bead and each lane's number of changes to have the parity of its end layers, and the triple rule orders $t_{23}$ before $t_{13}$. The solver slides every bead at once and proves the fewest changes.

Proof: Lanes that never change layer stay on F and so cannot cross one another; they form an increasing subsequence of $\pi$. Every other lane leaves F and returns, at least two vias. ∎

The floor is the classical maximum planar subset of the two-shore case [4], and it is weak: lanes that leave F also cross one another, and an excursion to B cannot serve a crossing with a partner that is itself on B. For a uniformly random permutation the LIS grows as $2\sqrt{K}$, with Tracy–Widom fluctuations of order $K^{1/6}$ [24]—the statistics of the KPZ class of growth processes [25]. Real buses are structured, but the scaling is a yardstick for how the floor grows. The next lemma counts a lane's vias exactly, given its partners.

Lemma 2 (Blocks): Fix the layers of lane $n$'s crossing partners at their crossings, in the order the crossings occur along $n$, and let both ends of $n$ be on F. The fewest layer changes of $n$ is twice the number of maximal runs of consecutive partners that are on F.

Proof: At each crossing $n$ must be on the partner's opposite layer: on B across a maximal run of F partners, on F across a run of B partners and at both ends. Each F run therefore needs one change into B and one out, and no change is needed elsewhere. ∎

The difficulty is therefore not placing a lane's vias given its partners, but choosing every lane's layers and crossing order together—which is what the whole-route model does.

## IV. The Whole-Route Solve

One CP-SAT [26] model decides every crossing and every layer change for all lanes at once (Fig. 2).

Crossings. For each $(a, b) \in I$ an integer $t_{ab} \in [\ell_{ab}, h_{ab}]$ on the grid $G$ is the crossing's position along the route. Its window starts after both lanes have entered the trunk, keeps a margin from both terminals, excludes the band in front of the source face where the teeth stand, and ends at the handoff to the rings unless both lanes share a ring. The triple rule: for every $i < j < k$ in launch order,

- if $ij, ik, jk \in I$: $t_{ij} < t_{ik} < t_{jk}$ or $t_{jk} < t_{ik} < t_{ij}$ (one Boolean);
- if $ij, ik \in I$ and $jk \notin I$: $t_{ij} < t_{ik}$;
- if $ik, jk \in I$ and $ij \notin I$: $t_{jk} < t_{ik}$.

The case $ij, jk \in I$, $ik \notin I$ cannot occur. The rule is a known characterization, stated here in the form the model uses:

Proposition 1: A linear order of $I$ is the order in which the crossings of some wiring diagram (reduced word) of $\pi$ occur if and only if it satisfies the triple rule.

Sketch: In a wiring diagram two lanes cross only when adjacent, so after any prefix of crossings the lanes stand in the order of some permutation, and the prefix is its inversion set; conversely, if a prefix and its extension by one pair are both inversion sets, that pair is adjacent and can cross next. A set of pairs is an inversion set if and only if, on every triple, it is closed ($ij, jk$ in the set imply $ik$) and co-closed ($ik$ implies $ij$ or $jk$); requiring this of every prefix, triple by triple, gives exactly the three cases above, which is Papi's characterization of convex orders [22]. ∎

These are the convex (reflection) orders of the inversion set [22]; for the longest permutation their commutation classes form the higher Bruhat order $B(n, 2)$ [23]. Crossings of disjoint pairs may coincide—they commute—while crossings that share a lane are kept apart by a per-lane no-overlap constraint on intervals of the lane pitch. A Boolean per crossing chooses which lane stays and which moves across: the stayer's interval is the full pitch, the mover's the shorter footprint of its sweep. The two crossings of a pair lane that pass other lanes in opposite directions are kept a turning run apart.

Worldlines. Read $u$ as time and the lane order as position on a line: each lane is then the worldline of a particle in one space and one time dimension, and a crossing is a two-body scattering event. Three mutually crossing lanes can scatter in only two orders, $t_{ij} < t_{ik} < t_{jk}$ or its reverse—the two sides of the Yang–Baxter relation [27], or the third Reidemeister move. The triple rule's Boolean is the choice between them.

Layer changes. Each lane $n$ has up to three change positions $c_{n,1..3}$ with activity literals $a_{n,k}$; active changes are ordered and a via's room $D_v$ apart, and their parity is fixed by the ends:

$$\bigoplus_k a_{n,k} \;=\; \ell_n^S \oplus \ell_n^D, \tag{1}$$

where $\ell_n^S, \ell_n^D \in \{0,1\}$ are the end layers. For each crossing $(a,b)$ and each change $k$ of lane $a$, a literal $\beta_{a,k,ab}$ states that the change lies before the crossing, enforced with clearance: $c_{a,k} \le t_{ab} - h$ or $c_{a,k} \ge t_{ab} + h$, where $h$ is the room of both lanes there (wider when the crossing lane is a pair, and dependent on the stay/move choice). The layer of $a$ at the crossing is $\ell_a^S \oplus \bigoplus_k (a_{a,k} \wedge \beta_{a,k,ab})$, and crossing lanes must differ:

$$\ell_a^S \oplus \ell_b^S \oplus \bigoplus_k (a_{a,k} \wedge \beta_{a,k,ab}) \oplus \bigoplus_k (a_{b,k} \wedge \beta_{b,k,ab}) \;=\; 1. \tag{2}$$

Changes of lanes that can be neighbors—adjacent in $\lambda$ or $\varphi$, or crossing each other—are staggered along $u$ by $\max(v, \sqrt{v_v^2 - p^2})$, so that two vias one pitch $p$ apart across the bus still meet the via-to-via rule $v_v$.

Spins. Give each worldline an Ising spin, its layer. Equation (2) is then a hard antiferromagnetic bond at every crossing, a via is a domain wall along a worldline, and the objective counts domain walls. A cycle of must-differ constraints with an odd number of links cannot be satisfied without a wall—it is frustrated [28]—and frustration is what forces vias. With the crossing order fixed this is the planar spin-glass ground state that Barahona et al. solved as a maximum cut alongside via minimization [8], and Lemma 2 counts the domain walls of one worldline in the field of its partners. The whole-route solve also chooses the order of the scattering events.

Built-in cuts keep a lane's changes off stretches within a via's reach of another part's pad or a foreign tooth, and a pair's off its known corners and its end connectors (Section VI-C).

Objective. With $o_n \ge 0$ and $o_n \ge s_n + \sum_k a_{n,k} - 2$ counting the vias of lane $n$ beyond two ($s_n$ its stub vias, the larger leg's for a pair),

$$\min\; W_O \sum_n o_n \;+\; W_V \sum_{n,k} a_{n,k} \;+\; H, \tag{3}$$

with $W_V = 10^6$, $W_O = W_V(3M+1)$ for $M$ lanes, and $H$, the history term of Section VI-E, below one via. Preferring at most two vias per net is a preference, not a cap: a cap is infeasible on many instances.

Proving it. CP-SAT runs four workers: two linear-relaxation searches, a core-based search and an objective lower-bound search. The bound-raising workers are essential: CP-SAT's default portfolio left the first solve of the 51-net rung unproved after 421 s (56 vias against a bound of 40); with them it is proved in 22–37 s. When they cannot prove a plan, a second run with plan-finding workers starts from the best plan and with the bound as a constraint. A re-solve in the loop only adds cuts and history below a via, so the first solve's optimum is a valid floor for it; adding the floor turned an unproved 193 s re-solve into a proof in 73 s. Every budget is a count of deterministic work, never a clock. Only a plan proved optimal in its vias goes on; one the search could not prove is no plan.

Scope. The proof is relative to the model: the ends are fixed, each inverted pair crosses once (non-reduced braids are excluded), a lane changes layer at most three times (every proved plan we have seen uses at most two), positions lie on $G$, and the loop's cuts are heuristic. What is proved is the fewest layer changes for the given ends. The board's via count also depends on the ends, whose stub vias are about half of it, and a better choice of ends can beat a proved plan: the 44-net Zynq rung routes in 58 vias on a superset of the 42-net rung's nets, where the 42-net plan uses 62.

## V. Choosing the Ends

Because the ends decide the braid, the escapes at both arrays are chosen with an estimate of the solve's objective, to which the search adds prices of its own: the lanes' length at a fixed rate per via, trunk congestion and crossings, and the audits' feedback. For a candidate assignment of teeth and berths we estimate the route's vias from the ends alone: among lanes whose ends share a layer, inverted pairs on one layer must be separated by lanes that dive, and since their conflict graph is a permutation graph the cheapest cover is the complement of a maximum-weight increasing chain, exact per layer; a lane whose ends differ changes once, and the partners it would meet on the wrong layer form a bipartite graph covered exactly by a minimum cut; conflicts between such lanes are resolved greedily. Congestion adds each lane's load on the trunk (the room its crossings and changes take), priced by the square of its excess over half, and a fifth of a via per trunk crossing. The crossing price exists because the solve's effort follows its crossings, not its vias: the 35-net rung proved in 10 s at 131 crossings and in 121 s at 163. The search moves one tooth or berth at a time (ejecting one conflicting lane where needed), then iterates from seeded random kicks; the best few states are re-ranked by the solve's order model without lengths, within a work budget, so the chosen ends are not proved best.

Street berths. A ball beside the unpopulated band of a DDR3 package may dive in a lane of the band and leave along it on the other layer, as the human does with 20 of its 30 signal vias at the DRAM. Offering such berths cut the 35-net rung's crossings from 162 to 132 and its solve from 121 to 10 s, at 6 more vias.

## VI. From Plan to Copper

### A. Geometry

A single linear program places every lane: an offset per lane per column, columns every four grid steps (0.1 mm) along the trunk and both rings, joined where each lane's trunk ends. Constraints keep each column's lanes in the solve's order on their solved layers, same-layer neighbors a bar apart (slope-corrected by tangent cuts), a via's room around every change (chords of a disc at neighboring columns), every lane inside the board and off the pad

boxes, and turns within 45° per column (per turning run for a pair). Costs are the added length—under-estimated by tangent cuts, so that a small sideways move costs next to nothing and lanes take a comfortable pitch where there is room—the bends, and a hinge penalty on neighbors closer than the comfortable pitch. Every hard constraint except the terminal pins, the end holds and the joins between trunk and rings is elastic, its slack priced at $W_H = 10^4$ per millimeter, so the program always has a solution and reports exactly where it had to break a rule.

A second pass holds each lane to one side of every island—a part's pads, or several parts no lane can pass between. The choice of side is non-convex, so it is made combinatorially before the pass, by neither the LP nor the CP-SAT model: at each island the lanes, in the order the CP-SAT solve fixed there, are split at the index that the first pass's offsets make cheapest, and the island is inserted into the order at that split. The sides are thus chosen consistent with the crossing plan, and the program stays convex; the polish may still flip a lane it cannot hold on its side (Section VI-B). The constraints paid in the second pass become cuts for the solve: a lane that could not be kept off an island may not cross anyone over the island's span; a change that could not be given its room may not lie there.

The program is large: pass one of the 51-net rung has 7 205 lane offsets, a slack or cost variable for each elastic constraint, and 150 960 constraints. We pass its dual to HiGHS [29] through SciPy and read the offsets back as the multipliers of the dual's balance equations, falling back to the dual simplex when the interior point does not terminate. The dual solves the two passes of the 28-net rung in 8 and 20 s, against 24 and 87 s for the primal by the same interior-point method and 18 and 48 s by HiGHS's default method; on the 51-net rung, in 23 and 53 s against 84 and 354 s and 153 and 567 s, each time at identical objective values.

Forces. The program's optimality conditions are those of a mechanism at rest (Fig. 3). Each lane point is a bead that slides on its column's rail, the length term is a string at unit tension, and the rules are contacts: struts between neighbors, walls at the pads and the board edge. A rule's dual value is the force in its contact, and complementary slackness is the contact law—a strut pushes only while it touches. An elastic rule is a rigid-plastic strut that carries up to $W_H$ and then yields. The static and kinematic theorems of plastic collapse are this duality [30], and the linear program that tests a packing of hard spheres for jamming solves for the same contact forces [31]. The picture is useful for reading a plan. In pass one of the 51-net rung only 376 of the hard constraints carry a force; together with the pinned ends and the length, bend and comfort terms, they hold every lane where it is, and they name the rules that do. A hard constraint the program could not keep has its elastic slack paid—a strut that yields—and those of the static, via and dive kinds are what the geometry returns to the solve as cuts.

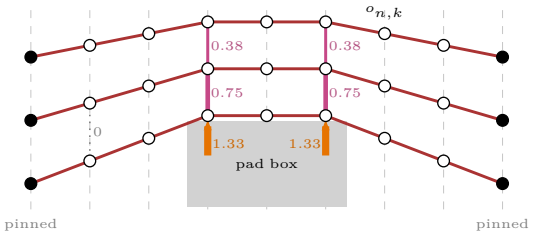



Fig. 3. A small instance of the geometry LP, solved. Three lanes on one layer cross nine columns, pinned at both ends (black), and must pass over a pad box. Each lane point slides on its column's rail (dashed); the lanes are strings at unit tension. The rules that bind carry the dual values drawn: the box (orange) pushes the lowest lane up where the lanes bend, and pitch struts (magenta) pass the load up the stack, each carrying the load of the lanes above it (their tension and bending; the toy also pulls each lane weakly toward its straight reference). Every other rule carries no force, like the strut near the pins (dotted), which has room to spare. Forces are in units of the string tension.

## B. Polish and snap

The polish meets the audit's own measures in board coordinates (pitch, via room, clearance to pads as drawn, turn shapes) by small vertex moves, one linear program per round, with every bar a grid step above the snap's. A lane that cannot be held on its side of an island is flipped, and the geometry runs again.

The snap makes the smooth plan octilinear on the router's grid, one lane at a time, by an A* search in a strip around each lane's smooth line that prices length, bends and distance from the line. Pairs go first, moving exactly as the pair router moves (45° turns, a straight run after each turn, a via only on a straight run); they are then held while the polish fits the singles around them. A lane not yet laid keeps a share of every gap—the side of the midline nearer its own line, less half a bar—so lanes laid first cannot take the room later ones need. Two sweeps then lay every lane again against the others' real copper.

## C. Differential pairs

A pair is one lane of two slots' width. Its hand at an end is the side of its travel on which P lies. A pair whose hands at its tooth and berth disagree must swap its legs somewhere; the solve then requires it to change layer at least once, and the snap lays a crossover at its first dive, as a designer would: the first leg jogs out at 45° to its barrel and dives, the second jogs at 45° over the first leg's new-layer segment to a barrel just beyond and returns to its line. Both barrels lie on one side of the pair, staggered by the fewest whole grid steps that keep a via's pitch and each jog's clearance, and every stage knows the crossover's footprint before it is laid. At each end the legs converge, through an end connector—two legs from the pair's tips, turning at most 45°—onto a pose, a grid point and heading from which the pair router takes over, so plan and router share one end.

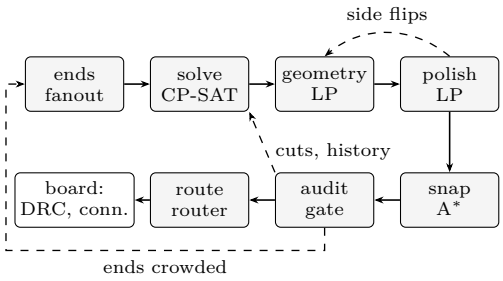



Fig. 4. The whole route. Solid arrows pass the plan forward; dashed arrows are the negotiation: findings between the arrays return to the solve as cuts and history, lanes with no room beside an island return to the geometry as side flips, and findings at the ends return to the choice of escapes.

### D. Audit and route

The audit checks the plan against the router's rules as the router will see them (pads as drawn, each terminal joined as the router joins it), and the gate passes it only when it is complete and clean with nothing waived. The production router [1] then lays the pairs, then the singles, each lane confined to its band—the router cells within half a grid step of its snapped line, with its end and via cells; independent checks grade connectivity and DRC on the written board.

### E. The negotiation loop

Every stage is fed a measurement of the one before (Fig. 4). The audit's findings return to the solve as history: every place a round found the plan short is priced, as a negotiated router prices a place overused before [19], by how many audits found it so—below the price of a via, so history reorders plans of equal vias but never buys one. The geometry's paid constraints and the snap's failures return as cuts; side flips return to the geometry. The loop stops when the gate passes, after two rounds that do not improve on the best, or when a finding stands at the ends, which only the fanout can move: the finding is then priced in the ends model and the next fanout round chooses again, from the previous round's board with only the named teeth free. This is a counterexample-guided loop in the spirit of combinatorial Benders decomposition [20], [21]; unlike it, a cut here generalizes one geometric failure and is not proved valid, so the loop's result is optimal only for the model it ends with.

Sectors. The route's non-convexity lives entirely in discrete choices—the crossing order and the side of each island, together a homotopy class of the whole bus. Within one such topological sector, and with the first pass's linearization fixed, the geometry is convex, and a small tie-break picks one point of its face of optima. The loop is a search over sectors: the solve proposes one, the geometry relaxes within it, and the constraints that yield tell the solve which sectors to leave.

TABLE I
Vias / copper (mm) of the run's nets on the finished board, and the whole route's wall time and peak memory on an 8-core laptop running four rungs side by side (so wall time includes waiting). Crossings are those of the solve that was routed. Every entry of the whole route is connected and DRC-clean at 0.1 mm.

| rung | whole route | human | crossings | wall (s) | MB |
|---|---|---|---|---|---|
| Allwinner H3 to DDR3 | | | | | |
| K15 | 10 / 199 | 22 / 232 | 6 | 39 | 241 |
| K28 | 30 / 589 | 48 / 678 | 57 | 169 | 436 |
| K35 | 58 / 779 | 60 / 889 | 141 | 341 | 434 |
| K41 | 70 / 1040 | 70 / 1081 | 207 | 595 | 593 |
| K51 | 80 / 1220 | 88 / 1337 | 281 | 1295 | 720 |
| Zynq-7020 to DDR3 | | | | | |
| K18 | 12 / 466 | 45 / 662 | 14 | 139 | 404 |
| K26 | 30 / 732 | 57 / 923 | 67 | 412 | 464 |
| K32 | 46 / 940 | 74 / 1166 | 87 | 475 | 693 |
| K38 | 54 / 1121 | 86 / 1373 | 118 | 428 | 671 |
| K42 | 62 / 1263 | 97 / 1534 | 167 | 1892 | 842 |
| K44 | 58 / 1347 | 103 / 1617 | 193 | 767 | 769 |

## VII. Results

### A. Benches and grading

H3. An Allwinner H3 (0.65 mm ball pitch) to a DDR3 device on a public reference design [32], whose designer routed the bus on its two layers. Its bench carries 51 bus nets, among them three differential pairs (two data strobes and the clock); the ladder's rungs K15–K51 add whole groups of nets, tightest first. Zynq. The Zynq-7020 (CLG400) and its DDR3 on a public design [33], whose designer routed the bus on the outer layers only (the inner layers are planes), 103 vias, every one F to B. We strip its routing, fan out the processor, and keep the two outer layers; the ladder runs to 44 nets with both strobe pairs. The numbers are for the bench as first built; the bench builder's net selection has since changed, so it is the bench file, not the build recipe, that reproduces them.

Each result is written as a board and graded independently: connectivity of every bus net, DRC at the routed clearance of 0.1 mm, and a census of every via and millimeter of the run's nets, stubs included. The human's copper on the same nets is counted the same way; it includes length-matching meanders, which add copper but no vias. Runs used CP-SAT from OR-Tools 9.15 and HiGHS through SciPy 1.16 on an Apple M3 laptop (8 cores, 8 GB).

### B. Against the human

Table I gives every rung of both ladders on our own ends. Every rung routes every lane in its band at once, every net connected, DRC-clean, never with more vias than the human and always with less copper. At the top of the H3 ladder the route uses 80 vias to the human's 88 (Fig. 5); on the Zynq, 58 to 103. Part of the copper difference is the human's length matching.

On the human's own ends at K51—its teeth and berths clipped from its board—the whole route uses 88 vias to the human's 88, in 1263 mm of copper to 1337. Our earlier approach, which routes corridors of lanes stage by stage

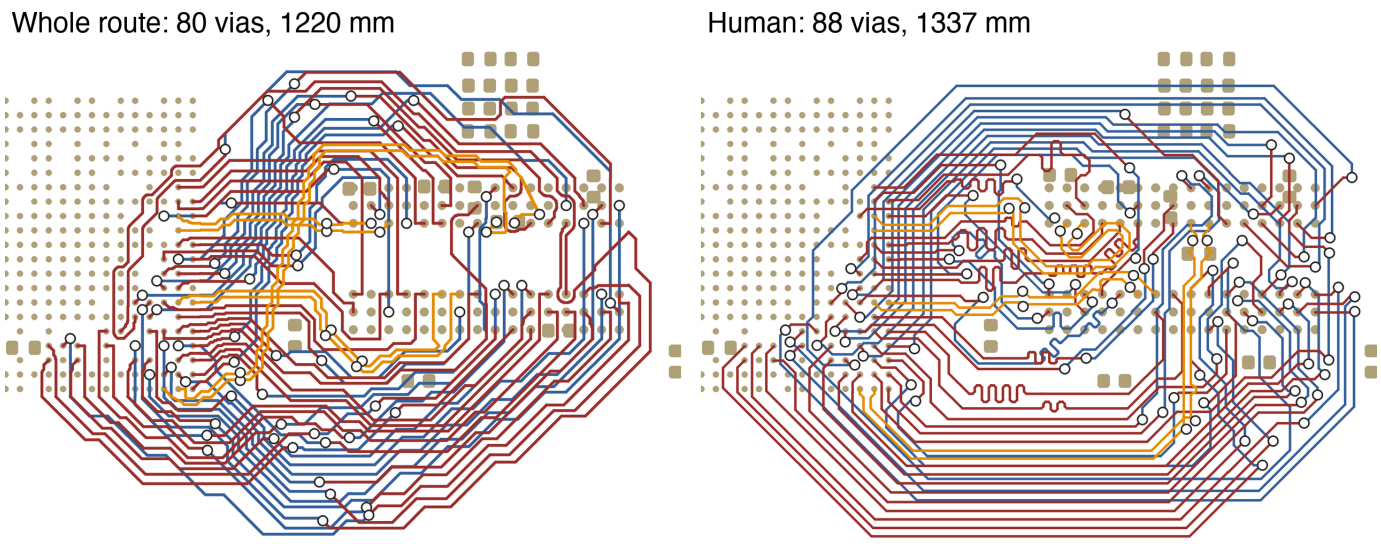



Fig. 5. The 51-net H3 rung, only the run's nets drawn. Left: our ends and the whole route, 80 vias and 1220 mm. Right: the human's board on the same nets, 88 vias and 1337 mm. F is red, B blue, the three differential pairs orange; every via is drawn as the same marker on both boards. The source BGA is at left, the DDR3 at right.

from a CP-SAT plan of the ends and repairs refusals by rip-up, needed 101 vias on those ends, and on our own ends left five nets open at 144 vias.

### C. Synthetic buses

The repository's generator plants a crossing pattern between two arrays and computes a reference optimum for the planted ends [1]. Over fifteen such buses—8, 15 and 28 lanes, in sorted, blocked, interleaved, riffled and reversed patterns—the whole route routes fourteen connected and DRC-clean, never above the reference and below it on ten. It gets below because it chooses its own ends, which the reference holds fixed: on the reversed 15-lane bus it uses 24 vias, fewer even than the floor of Lemma 1 for the planted permutation (28). These cases therefore measure what choosing the ends is worth, not the optimality of the solve. On the fully reversed 28-lane bus, whose 282 crossings all fall on the trunk, the solve found no plan within its work budget.

### D. Cost

Most of the time is spent choosing the ends: of K51's 1295 s, the fanout with its exact re-ranking takes 829 s, the solve 22 s, and one loop round under three minutes, half of it the geometry LP. The solve grows with its crossings (Table I) and their triples: at the top of the H3 ladder, 281 crossings and 2 324 triples are proved optimal in 22 s on the laptop and 81 s on a four-core Linux container, and no Zynq solve takes longer than 15 s there.

## VIII. Limitations and Future Work

The route does not match lengths; a DDR interface requires length (or delay) matching within byte lanes and of the address group to the clock [34], and the human's copper includes that work. Matching lengths on the planned lanes, where the comfortable pitch leaves room for meanders, is our next step. The model is for two signal layers; boards with more layers route the bus on more, and the model's spins become multi-valued. Optimality is relative to the model (fixed ends, reduced braids, at most three changes per lane, heuristic cuts), and the ends are chosen by a local search whose estimate is exact only in its parts; a single CP model over the ends could prove them. With the crossing order fixed and via room set aside, the layer assignment is a planar Ising ground state, a minimum T-join on the planar dual of the wiring diagram, and so solvable in polynomial time [7], [35]; it could give the ends search a fast exact evaluator and a lower bound. The solve counts a pair's dive once, where the board counts two barrels, so its equal optima can differ on the board by two vias. Intra-pair skew is not yet planned. Finally, two boards are few, and the design constants were developed on them; we are extending the evaluation to more boards and to synthetic buses with their ends held fixed, where the reference optimum applies.

## IX. Conclusion

Planning a bus whole—ends, crossings, their order and every layer change, decided at once, with the fewest layer changes proved for the chosen ends, and negotiated with the geometry until the production router can lay it—routes two real DDR3 buses fully automatically, with fewer vias than their designers. The central model is small and exact because the problem's combinatorics are those of a wiring diagram, whose consistency is a rule over triples. Throughout this work the synergies between routing and physics proved abundant, interesting and of practical use: the worldline picture gives the triple rule its meaning, the spin picture traces every via to

frustration, and the forces of the linear program's dual name the rules that hold each lane and the ones that must yield. We expect the same pairing—an exact choice of topological sector, relaxed in a convex statics problem whose forces steer the next choice—to carry over to length matching, to boards with more layers and to the other dense interconnects of modern hardware, and the dialogue between routing and physics to keep paying in both directions.

## Acknowledgment

In accordance with IEEE policy, the author discloses that portions of the software were drafted with the assistance of an AI system (Claude, Anthropic); the author reviewed and takes responsibility for all content.

## References

[1] A. Haas, "KiCadRoutingTools," [Online]. Available: https://github.com/drandyhaas/KiCadRoutingTools, 2026.
[2] C.-P. Hsu, "Minimum-via topological routing," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 2, no. 4, pp. 235–246, 1983.
[3] M. Marek-Sadowska, "An unconstrained topological via minimization problem for two-layer routing," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 3, no. 3, pp. 184–190, 1984.
[4] M. Sarrafzadeh and D. T. Lee, "A new approach to topological via minimization," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 8, no. 8, pp. 890–900, 1989.
[5] R. Y. Pinter, "Optimal layer assignment for interconnect," J. VLSI Comput. Syst., vol. 1, no. 2, pp. 123–137, 1984.
[6] N. J. Naclerio, S. Masuda, and K. Nakajima, "The via minimization problem is NP-complete," IEEE Trans. Comput., vol. 38, no. 11, pp. 1604–1608, 1989.
[7] F. O. Hadlock, "Finding a maximum cut of a planar graph in polynomial time," SIAM J. Comput., vol. 4, no. 3, pp. 221–225, 1975.
[8] F. Barahona, M. Grötschel, M. Jünger, and G. Reinelt, "An application of combinatorial optimization to statistical physics and circuit layout design," Oper. Res., vol. 36, no. 3, pp. 493–513, 1988.
[9] M. M. Ozdal and M. D. F. Wong, "Algorithms for simultaneous escape routing and layer assignment of dense PCBs," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 25, no. 8, pp. 1510–1522, 2006.
[10] M. M. Ozdal, M. D. F. Wong, and P. S. Honsinger, "Simultaneous escape-routing algorithms for via minimization of high-speed boards," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 27, no. 1, pp. 84–95, 2008.
[11] H. Kong, T. Yan, and M. D. F. Wong, "Automatic bus planner for dense PCBs," in Proc. 46th ACM/IEEE Design Automat. Conf. (DAC), 2009, pp. 326–331.
[12] J.-T. Yan, "Layer assignment of buses and nets with via-count constraint in high-speed PCB designs," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 38, no. 3, pp. 512–525, 2019.
[13] L. Luo and M. D. F. Wong, "Ordered escape routing based on Boolean satisfiability," in Proc. Asia South Pacific Design Automat. Conf. (ASP-DAC), 2008, pp. 244–249.
[14] S. Bayless, H. H. Hoos, and A. J. Hu, "Scalable, high-quality, SAT-based multi-layer escape routing," in Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD), 2016, pp. 1–8.
[15] T. Yan, P.-C. Wu, Q. Ma, and M. D. F. Wong, "On the escape routing of differential pairs," in Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD), 2010, pp. 614–620.
[16] H. Wu, X. Jiang, Y. Fan, X. He, and M. Liu, "Topology optimization-based layer assignment method for fixed-layer complex PCBs," IEEE Trans. Compon., Packag., Manuf. Technol., vol. 16, no. 1, pp. 20–31, 2026.
[17] W. W.-M. Dai, T. Dayan, and D. Staepelaere, "Topological routing in SURF: Generating a rubber-band sketch," in Proc. 28th ACM/IEEE Design Automat. Conf. (DAC), 1991, pp. 39–44.
[18] C. E. Leiserson and F. M. Maley, "Algorithms for routing and testing routability of planar VLSI layouts," in Proc. 17th ACM Symp. Theory Comput. (STOC), 1985, pp. 69–78.
[19] L. McMurchie and C. Ebeling, "PathFinder: A negotiation-based performance-driven router for FPGAs," in Proc. ACM Int. Symp. Field-Programmable Gate Arrays (FPGA), 1995, pp. 111–117.
[20] J. N. Hooker and G. Ottosson, "Logic-based Benders decomposition," Math. Program., vol. 96, no. 1, pp. 33–60, 2003.
[21] G. Codato and M. Fischetti, "Combinatorial Benders' cuts for mixed-integer linear programming," Oper. Res., vol. 54, no. 4, pp. 756–766, 2006.
[22] P. Papi, "A characterization of a special ordering in a root system," Proc. Amer. Math. Soc., vol. 120, no. 3, pp. 661–665, 1994.
[23] Y. I. Manin and V. V. Schechtman, "Arrangements of hyperplanes, higher braid groups and higher Bruhat orders," in Algebraic Number Theory – in honor of K. Iwasawa, ser. Adv. Stud. Pure Math., 1989, vol. 17, pp. 289–308.
[24] J. Baik, P. Deift, and K. Johansson, "On the distribution of the length of the longest increasing subsequence of random permutations," J. Amer. Math. Soc., vol. 12, no. 4, pp. 1119–1178, 1999.
[25] M. Prähofer and H. Spohn, "Universal distributions for growth processes in 1+1 dimensions and random matrices," Phys. Rev. Lett., vol. 84, no. 21, pp. 4882–4885, 2000.
[26] L. Perron and F. Didier, "CP-SAT," Google OR-Tools, v9.15. [Online]. Available: https://developers.google.com/optimization/cp/cp_solver/, 2025.
[27] R. J. Baxter, "Partition function of the eight-vertex lattice model," Ann. Phys., vol. 70, no. 1, pp. 193–228, 1972.
[28] G. Toulouse, "Theory of the frustration effect in spin glasses: I," Commun. Phys., vol. 2, pp. 115–119, 1977.
[29] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," Math. Program. Comput., vol. 10, no. 1, pp. 119–142, 2018.
[30] A. Charnes, C. E. Lemke, and O. C. Zienkiewicz, "Virtual work, linear programming and plastic limit analysis," Proc. R. Soc. Lond. A, vol. 251, no. 1264, pp. 110–116, 1959.
[31] A. Donev, S. Torquato, F. H. Stillinger, and R. Connelly, "A linear programming algorithm to test for jamming in hard-sphere packings," J. Comput. Phys., vol. 197, no. 1, pp. 139–166, 2004.
[32] hvwyl, "allwinner-h3-ddr3-16x1-kicad: Allwinner H3 with DDR3 16x1 design reference for KiCad," [Online]. Available: https://github.com/hvwyl/allwinner-h3-ddr3-16x1-kicad, 2023.
[33] kangyuzhe666, "ZYNQ7010-7020_AD9363: Open-source SDR hardware based on Zynq and AD9363," [Online]. Available: https://github.com/kangyuzhe666/ZYNQ7010-7020_AD9363, 2020.
[34] M. M. Ozdal and M. D. F. Wong, "A length-matching routing algorithm for high-performance printed circuit boards," IEEE Trans. Comput.-Aided Design Integr. Circuits Syst., vol. 25, no. 12, pp. 2784–2794, 2006.
[35] F. Barahona, "On the computational complexity of Ising spin glass models," J. Phys. A: Math. Gen., vol. 15, no. 10, pp. 3241–3253, 1982.